

Advanced Optimization

Chapter 9 — Derivative-Free Optimization

Hongseok Kim

Department of Electronic Engineering, Sogang University

Fall 2026

Reading: Nocedal & Wright, *Numerical Optimization*, 2nd ed., Chapter 9

Self-study deck — this chapter is not covered in lectures.

When you have no derivatives — and why differencing may not save you

Sources of derivative-free problems: black-box simulations, legacy or proprietary code, functions computed through inner solvers with loose tolerances, physical experiments. Common model:

$$f(x) = h(x) + \phi(x), \quad (9.2)$$

h smooth, ϕ **noise** (need not even be a function — a stochastic simulation returns different values at the same x).

Lemma 9.1 — finite differences meet noise

With noise level $\eta(x; \epsilon) = \sup_{\|z-x\|_\infty \leq \epsilon} |\phi(z)|$ and $\nabla^2 h$ Lipschitz:

$$\|\nabla_\epsilon f(x) - \nabla h(x)\|_\infty \leq L_h \epsilon^2 + \frac{\eta(x; \epsilon)}{\epsilon}. \quad (9.5)$$

This is Chapter 8's V-curve with a **noise floor**: when η dominates ϵ , the computed “gradient” has no accuracy at all — $-\nabla_\epsilon f$ is a descent direction only by luck.

Hence the chapter's two responses: spread sample points *wider* and build a **model** (§9.2, §9.6), or abandon gradients entirely and compare **function values along directions** (§9.3–9.5).

Map of the chapter

Family	Idea	Verdict (book's)
Model-based (§9.2)	interpolate a quadratic, trust region	most effective; costly setup
Coordinate search (§9.3)	cycle through $e_1, \dots, e_n$	simple; can fail to converge
Pattern search (§9.3)	frame of directions, shrink/expand	convergent with the right frame
Conjugate directions (§9.4)	conjugacy from line searches only	elegant; small n
Nelder–Mead (§9.5)	reflect/contract a simplex	popular; can stagnate
Implicit filtering (§9.6)	FD gradient with shrinking ϵ	for controllable noise

Read this chapter with two warnings in mind

- ▶ All of it is for **small** n (tens, maybe a few hundred) and expensive f .
- ▶ DFO is a *last* resort. If source code exists, Chapter 8's AD gives exact gradients — “I don't feel like deriving gradients” is not a DFO problem.

Interpolate a quadratic, trust the region (Algorithm 9.1)

Keep a sample set $Y = \{y^1, \dots, y^q\}$ containing the best point x_k . Build

$$m_k(x_k + p) = c + g^T p + \frac{1}{2} p^T G p \quad (9.6)$$

from the **interpolation conditions** $m_k(y^l) = f(y^l)$ (9.7). Unknowns: $\frac{1}{2}(n+1)(n+2)$, so uniqueness needs

$$q = \frac{1}{2}(n+1)(n+2) \quad (9.8)$$

points in “general position” (nonsingular system). Then: trust-region step (9.9), ratio ρ , and one of three branches:

Algorithm 9.1, in words

- ▶ $\rho \geq \eta$: accept; swap the trial point into Y ; possibly grow Δ .
- ▶ $\rho < \eta$, but Y 's geometry is fine: the model was trusted too far — shrink Δ .
- ▶ $\rho < \eta$ and Y is degenerating: **fix the geometry, not the radius** — replace a point of Y to improve the conditioning of (9.7).

The third branch is what distinguishes DFO trust regions from Chapter 4: failure may mean a bad *sample set*, not a bad radius — e.g. iterates collapsing onto a low-dimensional surface.

Keeping the sample set healthy, and keeping the cost sane

Geometry via Lagrange functions (keyword level)

With basis functions ϕ_i , well-posedness is $\delta(Y) = \det[\phi_i(y^j)] \neq 0$ (9.17). For each $y \in Y$ define the Lagrange polynomial $L(\cdot, y)$ ($= 1$ at y , 0 on the rest); swapping $y_- \rightarrow y_+$ scales the determinant by $|L(y_+, y_-)|$ (9.18). So: on acceptance, evict $y_- = \arg \max_y |L(x^+, y)|$; on geometry repair, insert the point that maximizes a Lagrange function over the ball. Y is “adequate” if no nearby swap can double $|\delta(Y)|$.

The cost problem

$O(n^2)$ evaluations *before the first step*; $O(n^4)$ work per iteration even with updating. Linear models ($G = 0$): only $n + 1$ points and $O(n^3)$ — but no curvature, slow.

Minimum-change updating — the quasi-Newton idea returns (9.19)

Keep only $\hat{q} \approx 2n + 1$ points and resolve the leftover freedom by

$$\min_{f, g, G} \|G - G_k\|_F \quad \text{s.t. } G \text{ symmetric, } m(y^l) = f(y^l),$$

— least change to the Hessian, exactly Chapter 6's principle with *function values* in place of the secant equation. An equality-constrained QP per iteration; productive steps from $O(n)$ points. This is the design behind Powell's NEWUOA/BOBYQA line — the state of the art.

Coordinate search: simple, intuitive, and not convergent

Cycle through $e_1, e_2, \dots, e_n$, line-searching each in turn.

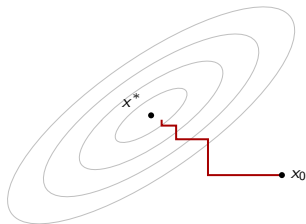


Figure 9.1: progress stalls in a rotated valley.

- ▶ Can iterate **forever without approaching a stationary point**, even with exact line searches —
 - $-\nabla f_k$ can become ever more perpendicular to the current coordinate direction ($\cos \theta_k \rightarrow 0$ with $\nabla f_k \not\rightarrow 0$; Zoutendijk is satisfied vacuously). True for cycling through *any* fixed independent set.
- ▶ When it does converge: usually slower than steepest descent, worse as n grows.
- ▶ Still used: no gradient needed, trivial to implement, acceptable when variables are *loosely coupled*. Variants: back-and-forth ordering; Hooke–Jeeves (add a search along the net cycle displacement).

(Where have we seen the fix for the rotated valley? Chapters 5 and 9.4: search along directions adapted to the function, not the axes.)

Pattern search (Algorithm 9.2): coordinate search done right

At x_k : evaluate f on a **frame** $\{x_k + \gamma_k p : p \in D_k\}$. Success ($f < f(x_k) - \rho(\gamma_k)$ for some p , with $\rho(t)/t \rightarrow 0$): move, possibly expand γ . Failure: stay, shrink $\gamma_{k+1} = \theta_k \gamma_k$. Stop when $\gamma_k \leq \gamma_{\text{tol}}$.

What makes it converge: conditions on the direction set

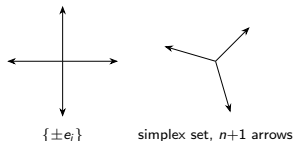
$$\kappa(D_k) = \min_{v \neq 0} \max_{p \in D_k} \frac{v^T p}{\|v\| \|p\|} \geq \delta > 0 \quad (9.21)$$

— *whatever* the (unknown!) gradient is, some frame direction makes an angle bounded away from 90° with $-\nabla f$; plus bounded lengths $\beta_{\min} \leq \|p\| \leq \beta_{\max}$ (9.22). Then some $p \in D_k$ has $-\nabla f_k^T p \geq \delta \beta_{\min} \|\nabla f_k\|$ — Chapter 3's $\cos \theta \geq \delta$ condition, achieved **without ever seeing a gradient**.

Examples with $\kappa > 0$ (Figure 9.2):

- ▶ $\{\pm e_1, \dots, \pm e_n\}$ — $2n$ points;
- ▶ simplex set $p_i = \frac{1}{2n} e - e_i$, $p_{n+1} = \frac{1}{2n} e$ — $n+1$ points.

Coordinate search = the case $D_k = \{\pm e_i\}$: $\kappa(D_k) = 0$. **That zero is the whole failure.**



Practical notes: accept the *first* sufficient-decrease point (no need to evaluate the whole frame); extra heuristic directions may be appended to the core set. Frames evaluate in parallel for free.

The parallel subspace property, and Powell's method

Parallel subspace property

Minimize the quadratic $f = \frac{1}{2}x^T Ax - b^T x$ over two parallel varieties $S_1 = x_1 + \text{span}\{p_1, \dots, p_l\}$ and $S_2 = x_2 + \text{span}\{p_1, \dots, p_l\}$, minimizers x_1^*, x_2^* . Then $x_2^* - x_1^*$ is **conjugate to every p_i** .

Proof is three lines: optimality gives $\nabla f(x_j^*)^T p_i = 0$, so

$$0 = (\nabla f(x_1^*) - \nabla f(x_2^*))^T p_i = (x_1^* - x_2^*)^T A p_i. \quad (9.26)$$

Conjugacy *observed* through line searches alone — no gradient, no A .

Algorithm 9.3 (sketch) and its safeguard

Cycle: n line searches along the current set $\{p_i\}$, then one along the net displacement $z - x_1$; that direction is conjugate to part of the set — swap it in, drop one, repeat. Quadratic f : minimizer in $\sim n$ cycles, $O(n^2)$ evaluations. Nonquadratic: interpolated line searches; the $\{p_i\}$ can drift toward *linear dependence* — Procedure 9.4 accepts a new direction only if it does not decrease $|\det(\hat{p}_1, \dots, \hat{p}_n)|$ (scaled $\hat{p}_i = p_i / \sqrt{p_i^T A p_i}$, computable from line-search byproducts).

Chapter 5 built conjugacy from gradients; this builds it from line searches. Same geometry, one fewer derivative.

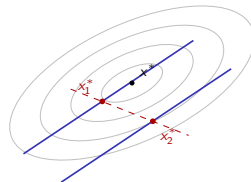
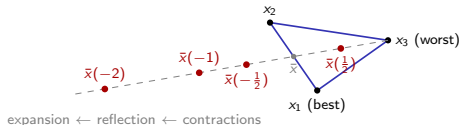


Figure 9.3: $x_2^* - x_1^*$ is conjugate to the common direction.

Nelder–Mead: the famous one

Keep a **simplex** of $n+1$ points, $f(x_1) \leq \dots \leq f(x_{n+1})$. Try to replace the *worst* vertex with a point on the line through it and the centroid $\bar{x}$ of the rest: $\bar{x}(t) = \bar{x} + t(x_{n+1} - \bar{x})$.



One step (Procedure 9.5)

Reflect ($t = -1$): if middling, take it. If best so far, try to *expand* ($t = -2$), keep the better. If still worse than x_n : *contract* outside ($t = -\frac{1}{2}$) or inside ($t = \frac{1}{2}$). If even that fails: *shrink* everything toward x_1 .

- ▶ Except in shrink steps, the *average* vertex value strictly decreases (9.29); convex f : shrink can't raise it either.
- ▶ Practical performance often reasonable; **stagnation at nonoptimal points happens** — restart when detected. Convergence theory: limited. (No relation to the LP simplex method.)
- ▶ This is `scipy.optimize.minimize(method="Nelder-Mead")` — know what you are getting.

Implicit filtering: differencing that keeps pace with the noise

For problems where the noise is *controllable* (simulation tolerance, number of trials): as the iterate improves, tighten the simulation and shrink the difference interval together.

Algorithm 9.6, in words

Outer loop over $\epsilon_k \downarrow 0$. Inner loop: steepest descent with the central-difference gradient $\nabla_{\epsilon_k} f$ and Armijo backtracking (Algorithm 3.1). Leave the inner loop — and shrink ϵ — when $\|\nabla_{\epsilon_k} f(x)\| \leq \epsilon_k$, or when $a_{\max}$ backtracks fail (the FD gradient is no longer trustworthy at this resolution).

Theorem 9.2 (flavor)

If $\nabla^2 fh$ is Lipschitz and the noise decays fast enough relative to $\{\epsilon_k\}$ (specifically $\eta(\cdot; \epsilon_k)/\epsilon_k \rightarrow 0$), then the FD gradients track ∇h and limit points are stationary for the smooth part h .

- ▶ The whole method is Lemma 9.1 turned into a schedule: keep ϵ above the noise floor, lower both together.
- ▶ Quasi-Newton flavors exist (BFGS on the filtered gradients).
- ▶ Right tool when *you* own the noise knob; wrong tool for irreducibly noisy f .

What to actually reach for (an opinionated page)

- 1 **First, question the premise.** Code available $\Rightarrow$ AD (Ch. 8) gives exact gradients; smooth and cheap $f \Rightarrow$ finite differences + BFGS beat everything in this chapter. DFO is for when derivatives are truly out of reach.
- 2 **Expensive, mildly noisy black box, $n \lesssim 100$:** model-based trust region — the minimum-change lineage (NEWUOA / BOBYQA). Book's verdict: most effective family.
- 3 **Noise you control:** implicit filtering.
- 4 **Need parallel, simple, provable:** pattern search with a $\kappa > 0$ frame.
- 5 **Quick look, tiny n , no stakes:** Nelder–Mead — and watch for stagnation.
- 6 *Beyond the book (2006):* CMA-ES for rugged landscapes, Bayesian optimization for very expensive f — today's defaults in ML hyperparameter tuning; same problem class, different machinery.

One connection to keep

Three ideas from the gradient world survive unchanged: trust the model only as far as it earns (Ch. 4), change the Hessian as little as the data demands (Ch. 6), and keep search directions from collapsing ($\kappa(D)$, $\det \hat{P}$ — the DFO version of $\cos \theta \geq \delta$).

Chapter 9 in one table, and optional exercises

Method	Evaluations/iter	Convergence	Noise?	Scales to
Model-based + min-change	$O(1)$ (+geometry)	to stationary pts	mild	$n \sim 10^2$
Coordinate search	~ 1 line search	may fail	poor	small
Pattern search	$\leq D_k $ (parallel)	yes, $\kappa > 0$	some	$n \sim 10^2$
Conjugate directions	n line searches/cycle	quadratic: finite	poor	small
Nelder–Mead	1–2 (+shrink n)	limited theory	some	$n \lesssim 20$
Implicit filtering	$2n$ per gradient	Thm 9.2	controllable	small

Optional exercises (if the chapter catches you)

- 1 Prove the parallel subspace property (9.26) and count the function evaluations Algorithm 9.3 needs on a quadratic.
- 2 Show $\kappa(\{\pm e_i\}_{i=1}^n) = 1/\sqrt{n}$ and $\kappa(\{\pm e_i\}$ used one axis at a time) = 0; reconcile with the convergence claims.
- 3 Run Nelder–Mead and BOBYQA (via `scipy.optimize`) on Rosenbrock with additive noise $10^{-4}\mathcal{N}(0, 1)$; compare with BFGS on finite differences. Lemma 9.1 predicts the loser — confirm it.

Back to the main track: Ch. 10–11 — *least squares and nonlinear equations*.